```
  def title?; @text.upcase == @text; end
  def reverse; @text.reverse; end
  def upcase; @text.upcase; end
  # ...
```

Paragraph objects are created in a single place in the Bookworm source code. Also, Paragraph#title? is called only once in the whole application, from a method named index:

```
def index(paragraph)
  add_to_index(paragraph) if paragraph.title?
end
```

Bill frowns. "The stupid Paragraph class really doesn't hold its own weight. We could scrap it entirely and just use regular Strings, if it weren't for the title? method."

"Why don't we *Monkeypatch (16)* the String class and add the title? method right there?" you offer. "I'm not convinced," Bill says. "A method with that name would make sense only on strings that represent a paragraph, not on each and every string."

While Bill is pondering the idea of patching the String class with a *Refinement (36)*, you decide to Google for a solution.

Introducing Singleton Methods

As it turns out, Ruby allows you to add a method to a single object. For example, here's how you can add title? to a specific string:

class_definitions/singleton_methods.rb

```
str = "just a regular string"

def str.title?
  self.upcase == self
end

str.title?                      # => false
str.methods.grep(/title?/)      # => [:title?]
str.singleton_methods           # => [:title?]
```

The previous code adds a method named title? to str. No other object gets the method—not even other Strings. A method like this one, which is specific to a single object, is called a *Singleton Method*. You can define a Singleton Method with either the syntax above or the Object#define_singleton_method method.

Spell: Singleton Method

Thanks to Singleton Methods, you can now fix your problem with the Bookworm source. You can send any old String to index if you enhance that String with a title? Singleton Method: